

DM Tools

Campaign Flow and Player Chronicle Plan

A maintainable campaign model built around changing world state, player agency, factual history, and shared memory.

DECISION	RECOMMENDATION
Core direction	Evolve DM Tools into a campaign-state addon. Model persistent pressures, actor goals, opportunities, and situations without predicting a session path.
Sessions	Do not plan them as DM records. Build a retrospective player chronicle from visible events, discoveries, and notes.
Delivery	Build the state model and migrations first, then dashboards and graphs, then player knowledge and optional integrations.

Architecture review and phased implementation plan | 26 July 2026

1. Executive plan

Recommended destination

DM Tools should describe the campaign at the level above any individual session: persistent pressures, actor goals, opportunities, and current situations. The host remains the durable source of truth for characters, places, events, mysteries, maps, backups, and synchronization; player session memory belongs with those visible facts.

What is already strong

- The host foundation is sufficient. DM-only collections, atomic multi-collection transactions, reviewed imports, lifecycle cleanup, graph facade, localization, and role-aware synchronization are already present.
- Core campaign entities are useful facts. Characters, factions, locations, events, mysteries, maps, and historical records should remain independent of planning tools.
- Addon boundaries are sound. DM Tools can work alone; sheets and compendiums can integrate through versioned APIs without becoming hard dependencies.

What currently blocks a coherent campaign flow

Gap	Why it matters	Design response
One flat scenario list	It cannot express persistent threats, independent actor goals, player opportunities, or changing situations.	Replace the flat catalog with an explicit campaign-state network.
Session is only an event number	This is suitable for history, but the player-facing recollection view is weak.	Derive a player chronicle from visible events and references; do not add DM session plans.
Scenario graph has nodes but no edges	It is a catalog in graph form, not a decision aid.	Render only explicit semantic links and scoped subgraphs.
Intent and history are easy to mix	Future edits can rewrite what happened or present one expected path as canonical.	Keep changing campaign state separate from canonical event history.
No persisted schema migration path	The data model is still small, so this is the cheapest moment to establish safe evolution.	Version records and add additive, tested migrations before new UI.

Scope guardrails

- No database, framework, or infrastructure rewrite. JSON remains appropriate at this campaign scale.
- No session plans, run sheets, expected scene order, or automatic debrief pipeline.
- No generic everything-links-to-everything graph. Use domain fields and a small link vocabulary.
- No combat engine inside DM Tools. A future encounter addon may optionally consume the planning API.
- No automatic publication of secret DM notes. Player-facing quests and chronicles use visible facts or an explicit projection.

2. Design around player agency

The DM maintains a world in motion, not a forecast of the next session. Player decisions select which parts of that state become relevant; consequences then reshape the state and the shared historical record.



The loop is state-driven: consequences reshape the world, not a prewritten next session.

Stage	Question	Tooling that should answer it	Durable result
Campaign state	What forces are active even if the party ignores them?	Fronts, actor goals, clocks, mysteries, opportunities, relationships, and locations.	A coherent world state.
Player choice	What do the characters choose to pursue now?	Visible hooks, known quests, prior discoveries, and ordinary campaign navigation.	No DM planning record is required.
Play	How does the relevant part of the world respond?	Existing campaign records plus quick access to linked situations and actors.	Improvised play remains unconstrained.
Consequences	What became true and which forces changed?	Create factual events and explicitly update only affected fronts, goals, quests, and mysteries.	Campaign facts and new state.
Shared memory	What do players know and remember?	Chronological session view derived from visible events, people, places, mysteries, and optional player notes.	A retrospective chronicle.

Critical modeling rule

No record should become invalid because the party skipped content, solved several problems at once, wandered in circles, or abandoned a quest. Sessions are labels on history, not containers for expected progress.

3. Proposed domain model

Use a network of persistent forces and current situations. Avoid a plot hierarchy, arbitrary recursive nesting, ordered scene chains, and any graph edge that predicts what the players will choose.

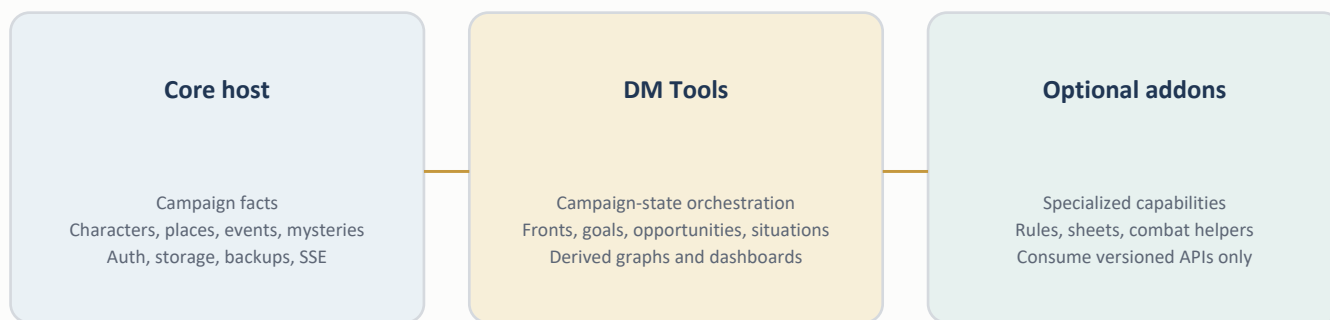


Concept	Owns	References	Lifecycle rule
Front / thread	Pressure, stakes, current state, clock or next escalation	Actors, factions, places, mysteries, opportunities	Persists independently of party attention; changes only when the world state changes.
Actor goal	Desired change, approach, progress, next pressure	Actor or faction, fronts, affected entities	May advance off-screen; never assumes a player response.
Opportunity / quest	Known goal, status, origin, result, public summary candidate	Fronts, situations, mysteries, people, places	May be pursued, combined, completed unexpectedly, ignored, or abandoned.
Situation / scenario	Current conditions, involved forces, stakes, triggers, possible consequences	Many fronts, opportunities, and core entities	A reusable description of what is true now, not an ordered encounter script.
Core event	What happened, when, where, who was involved	Core entities and affected campaign-state records	Canonical fact after play; never a prediction.
Session chronicle	Retrospective title, optional player notes, visible event grouping	Visible events and discovered entities	Player memory only; derived first and persisted only if notes prove useful.

Relationship discipline

- Use domain fields for ownership and involvement: actorIds, factionIds, locationIds, mysteryIds, frontIds, and opportunityIds.
- Use a very small explicit relation set only where fields are insufficient: supports, opposes, pressures, and related.
- Do not store leads-to, next-scene, or expected-session edges. Consequences may open new records after they become relevant.
- Tags aid filtering but never create graph edges. Timestamps, session numbers, and list order never imply narrative causality.

4. Tooling surfaces and ownership



Ownership is explicit. Integration is optional and contract-based.

Surface	Primary job	Show by default	Avoid
DM dashboard	Orient the DM in under 30 seconds.	Active fronts, advancing goals, unresolved mysteries, open opportunities, unstable situations, recent facts.	A next-session agenda, all records, or speculative analytics.
Front and goal board	Track what the world is doing.	Pressure, state, clocks, owners, affected people and places.	A prewritten story sequence or forced player path.
Opportunity / quest board	Track goals the party knows or could discover.	Status, origin, known facts, linked fronts and situations.	Assigning work to a session or assuming completion order.
Situation editor	Describe a playable state of affairs.	Conditions, actors, place, stakes, triggers, possible consequences.	Ordered scenes, scripted outcomes, or rules automation.
Consequence update	Record what became true.	Core event link plus only the fronts, goals, opportunities, and mysteries affected.	A mandatory session debrief or silent automatic changes.
Player chronicle	Help players remember and organize knowledge.	Visible events grouped by session, involved entities, discoveries, backlinks, optional notes.	DM-only plans, hidden alternatives, or guessed causality.
Campaign graph	Expose pressures and relationships.	One front, actor, quest, mystery, or location scope with list fallback.	Whole-campaign graph as the default.
Import center	Bring in structured campaign state safely.	Schema diagnostics, exact preview, atomic commit.	A second editing workflow or implicit overwrites.

Navigation recommendation

- Keep the stable /dm dashboard route as the entry point.
- Add DM routes for fronts, opportunities/quests, and situations; let each page deep-link to filtered graph views.
- Place session history in the host's player-visible campaign navigation, derived from visible events rather than registered by DM Tools.
- Treat the graph as a view mode within a domain page, not a separate source of truth.
- Keep core entity navigation in the host; DM Tools links into it using existing routes and collection descriptors.

5. Storage, contracts, and safe evolution

The main architectural risk is not storage volume. It is allowing the campaign-state model to grow without clear ownership, migration, and publication rules.

Collection	Shape	Access	Notes
fronts	Keyed	DM-only	Persistent pressure, state, stakes, clocks, and affected entities.
actorGoals	Keyed	DM-only	Owner, desired change, progress, approach, and next pressure.
opportunities	Keyed	DM-only initially	Quest semantics without scheduling; prepare for explicit public projection.
scenarios	Keyed	DM-only	Migrate from the current list shape; use situation semantics and preserve stable IDs.
stateLinks	List or keyed edges	DM-only	Only supports, opposes, pressures, and related relations; validate every endpoint.

Record evolution

- Add schemaVersion to persisted planning records before introducing new shapes.
- Ship additive, deterministic migrations with fixtures for old, mixed, partial, and repeated-startup states.
- Keep import schema versioning separate from persisted schema versioning. Imports describe input; migrations protect stored campaign data.
- Use host transactions for multi-collection consequences and migration publication.
- Centralize domain validation and status definitions so dashboard, graph, editor, server provider, and tests cannot drift.

Visibility and publication

Player memory must be built from player-visible truth

Derive session chronicles from visible core events and references. If players later need persistent summaries or quest cards, add a separate public record or explicit publish snapshot that copies only approved fields. Never expose DM pressures, hidden goals, undiscovered opportunities, or alternate consequences.

Integration contract

- DM Tools remains independently useful and has no hard dependency on sheets or the compendium.
- Expose a small versioned read API only after the model stabilizes: public opportunity summaries, active state counts, and linked core IDs.
- Optional consumers may enrich views but must degrade cleanly when absent. DM Tools never owns ruleset, combat, or character-sheet calculations.

6. Scalability, integrity, and maintenance

At expected campaign sizes, JSON and browser rendering are sufficient. Semantic and UI scale will fail before storage scale, so the plan emphasizes scoping, indexing, and diagnostics.

Concern	Recommendation	Reason
Graph growth	Default to front, actor, opportunity, mystery, or location scope; paginate/filter list fallback; never mount the whole campaign automatically.	The facade limit is generous, but a readable graph usually fails far below 1,000 nodes.
Dashboard growth	Show active pressure, changed, stalled, and unresolved slices plus counts. Link to full pages.	Rendering every situation becomes slower and less useful as the campaign grows.
Change detection	Use collection revisions or a deterministic content digest, not counts plus newest timestamp.	Equivalent counts/timestamps can hide meaningful edits.
Reference integrity	Add read-only diagnostics for dangling IDs, duplicate IDs, invalid states, broken state links, and invisible chronicle references.	The current local dataset already contains a dangling location reference.
Location semantics	Separate semantic containment from map-placement context when local-map work resumes.	A single parent field cannot safely express both relationships.
Temporal fields	Separate user/source time from host audit revision where conflicts or ordering depend on it.	One updatedAt value should not simultaneously mean provenance, sort order, and concurrency.
Graph fallback	Keep the accessible list usable when Cytoscape is missing, unsupported, or fails.	Graph availability must not remove core planning functionality.

Operational invariants

- Every ID is stable across imports, migration, reinstall, backup, and restore.
- Every cross-record reference is validated on write and reported on read if legacy data is inconsistent.
- Every multi-record consequence update is atomic: either all selected updates publish or none do.
- Every derived graph can be recreated from persisted records; graph layout is presentation state only.
- Every session chronicle is reproducible from visible events; missing or irregular session numbers degrade into an ungrouped history view.
- Every optional integration has a usable absent-provider state.
- Every state transition has an explicit user action and a localized failure path.

Deferred until demanded by real play

- Arbitrary recursive story trees, custom user-defined link types, automated quest progression, session plans, and predictive analytics.
- A full encounter/combat subsystem, initiative tracking, or rules adjudication inside DM Tools.
- Structured clue delivery beyond the existing mystery model, unless at-table use proves that free-form clues are insufficient.
- Server-side graph layout persistence and collaborative cursor state.

7. Phased implementation plan

The order is intentionally dependency-driven: define and protect the model before building screens that would freeze accidental semantics.

1

Domain foundation and migration

Build: Canonical status/validation module; schemaVersion; keyed scenarios; fronts, actorGoals, opportunities, stateLinks; deterministic migrations; integrity diagnostics.

Gate: Old data survives restart and backup/restore; invalid references are reported; no UI behavior regresses.

2

Campaign-state workflows

Build: Front, actor-goal, opportunity, and situation pages; explicit references; focused filters; restrained transitions; import schema update with exact preview.

Gate: A DM can model one real conflict without predicting player choices or using raw JSON.

3

State dashboard and scoped graph

Build: Active pressures, changed and stalled records, unresolved mysteries, scoped graph views, independent list fallback, revision-based refresh.

Gate: The dashboard answers what is happening in the world, not what should happen next session.

4

Consequences and factual history

Build: Optional event backlinks and atomic multi-record updates through F2 for affected fronts, goals, opportunities, situations, and mysteries.

Gate: Unexpected outcomes are recorded without reparenting content or satisfying a planned sequence.

5

Player session chronicle

Build: Core player-visible history grouped retrospectively by event session number, with involved entities, discoveries, backlinks, and optional notes only if needed.

Gate: Players can reconstruct past play; DM-only state and hidden alternatives never appear.

6

Optional integrations and player projection

Build: Versioned campaign-state read API; optional encounter/sheet enrichments; explicit public quest-card publishing if desired.

Gate: Removing any optional addon leaves DM Tools functional; published player data contains no secret fields.

8. Decisions, tradeoffs, and acceptance criteria

Decision	Recommendation	Tradeoff accepted
Model shape	Use a network of fronts, actor goals, opportunities, and current situations.	It cannot encode a prewritten plot tree, but it handles divergence and simultaneous consequences naturally.
Session relationship	A session is a retrospective grouping of visible history, never a DM planning record.	The DM gets no dedicated run sheet, but campaign state remains reusable and never goes stale when players diverge.
State vs fact	Keep changing pressures and possibilities in DM Tools and actual outcomes in core events.	Updating consequences remains deliberate, preventing speculation from becoming history.
Graph storage	Persist semantic links only; derive nodes, labels, grouping, and layout.	Views may require recomputation, but data remains portable and deterministic.
Player quests	Start DM-only; publish explicit safe projections later.	The first release lacks a player quest journal, but avoids accidental secret leakage and coupled schemas.
Integration	Soft, versioned, capability-based APIs only.	Some enrichment is unavailable when providers are absent, but each add-on remains maintainable and independently useful.

Release acceptance criteria

- Understandable: a new maintainer can explain the difference between a front, actor goal, opportunity, situation, event, and session chronicle without reading implementation code.
- Recoverable: migration, transaction failure, add-on disable/re-enable, restart, backup, and restore preserve campaign data and stable IDs.
- Agency-safe: skipping, combining, unexpectedly resolving, or abandoning opportunities never invalidates a plan or forces record reorganization.
- Traceable: a factual event can identify which fronts, goals, opportunities, situations, or mysteries it affected without claiming that it was expected.
- Memorable: players can navigate visible events by session and follow backlinks to the people, places, and discoveries they knew at the time.
- Scoped: dashboards and graphs remain useful as data grows because they open on active, recent, or selected subsets.
- Optional: DM Tools works without sheets, compendium, or future combat tooling; integrations fail closed and degrade visibly.

Recommended immediate next batch

Implement Phase 1 only. Establish fronts, actor goals, opportunities, situation semantics, record schemas, migrations, centralized validation, and integrity diagnostics before changing the dashboard or graph. Do not add a session-plan collection or session-focused DM workflow.

Review basis

This plan is based on a source-level audit of the host and DM Tools contracts, current planning/import/graph/dashboard implementations, reference documentation, and the local campaign data shape. It does not sample either remote production campaign. No campaign names, prose, record IDs, or personal data are included.